

Lab 2 — Task 5: Deploy an Application to Kubernetes (Minikube)

Akash Singh Sagar | Repository: github.com/akashsinghsagar/devops-lab2

Objective

Deploy the containerized Flask application (lab2-python-app:1.0) to a local Minikube cluster using Deployment and Service manifests, then verify that the pods are running and that the application is reachable through the Service.

Step 1 — Start the Minikube cluster

Minikube provides a single-node Kubernetes cluster on the local machine. It is started with the Docker driver, and the cluster and node are checked before anything is deployed.

```
minikube start --driver=docker
minikube status
kubectl cluster-info
kubectl get nodes
```



```
Problems Output Terminal Ports Spell Checker
Terminal
C:\Users\ARSH\OneDrive\Desktop\PDFs\zeel\labssss\lab2-k8s-kit>minikube start --driver=docker
minikube v1.39.0 on Microsoft Windows 11 Home Single Language 25H2
Using the docker driver based on existing profile
Starting "minikube" primary control-plane node in "minikube" cluster
Pulling base image v0.0.51 ...
> index.docker.io/kicbase/sta...: 18.17 MiB / 507.61 MiB 3.58% 1.55 MiB p Downloading Kubernetes v1.37.0
preload ...
> index.docker.io/kicbase/sta...: 507.61 MiB / 507.61 MiB 100.00% 3.24 Mi
> preloaded-images-k8s-v18-v1...: 150.33 MiB / 347.58 MiB 43.25% 1.53 MiB ! minikube was unable to downloa
d gcr.io/k8s-minikube/kicbase:v0.0.51, but successfully downloaded docker.io/kicbase/stable:v0.0.51 as a fallback
image
> preloaded-images-k8s-v18-v1...: 347.58 MiB / 347.58 MiB 100.00% 1.28 Mi
docker "minikube" container is missing, will recreate.
Creating docker container (CPUs=2, Memory=3900MB) ...
! Failing to connect to https://registry.k8s.io/ from both inside the minikube container and host machine
! To pull new external images, you may need to configure a proxy: https://minikube.sigs.k8s.io/docs/reference/n
etworking/proxy/
Preparing Kubernetes v1.37.0 on containerd 2.3.4 ...
Configuring CNI (Container Networking Interface) ...
Verifying Kubernetes components...
  Using image gcr.io/k8s-minikube/storage-provisioner:v5
  Enabled addons: storage-provisioner, default-storageclass
! C:\Program Files\Docker\Docker\resources\bin\kubectl.exe is version 1.31.4, which may have incompatibilities
with Kubernetes 1.37.0.
  Want kubectl v1.37.0? Try 'minikube kubectl -- get pods -A'
Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default

C:\Users\ARSH\OneDrive\Desktop\PDFs\zeel\labssss\lab2-k8s-kit>minikube status
! Executing "docker container inspect minikube --format={{.State.Status}}" took an unusually long time: 2.22405
5s
! Restarting the docker service may improve performance.
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

C:\Users\ARSH\OneDrive\Desktop\PDFs\zeel\labssss\lab2-k8s-kit>kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:64284
CoreDNS is running at https://127.0.0.1:64284/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
```

Figure 1 — Minikube is running; the single node minikube reports STATUS "Ready" and kubectl is pointed at the cluster.

Step 2 — Make the image available to the cluster

The Minikube node keeps its own image store, separate from the host's Docker images. The locally built image is therefore loaded into the node, and the Deployment uses `imagePullPolicy: IfNotPresent` so Kubernetes uses it instead of trying to pull from a registry.

```
minikube image load lab2-python-app:1.0
minikube image ls
```

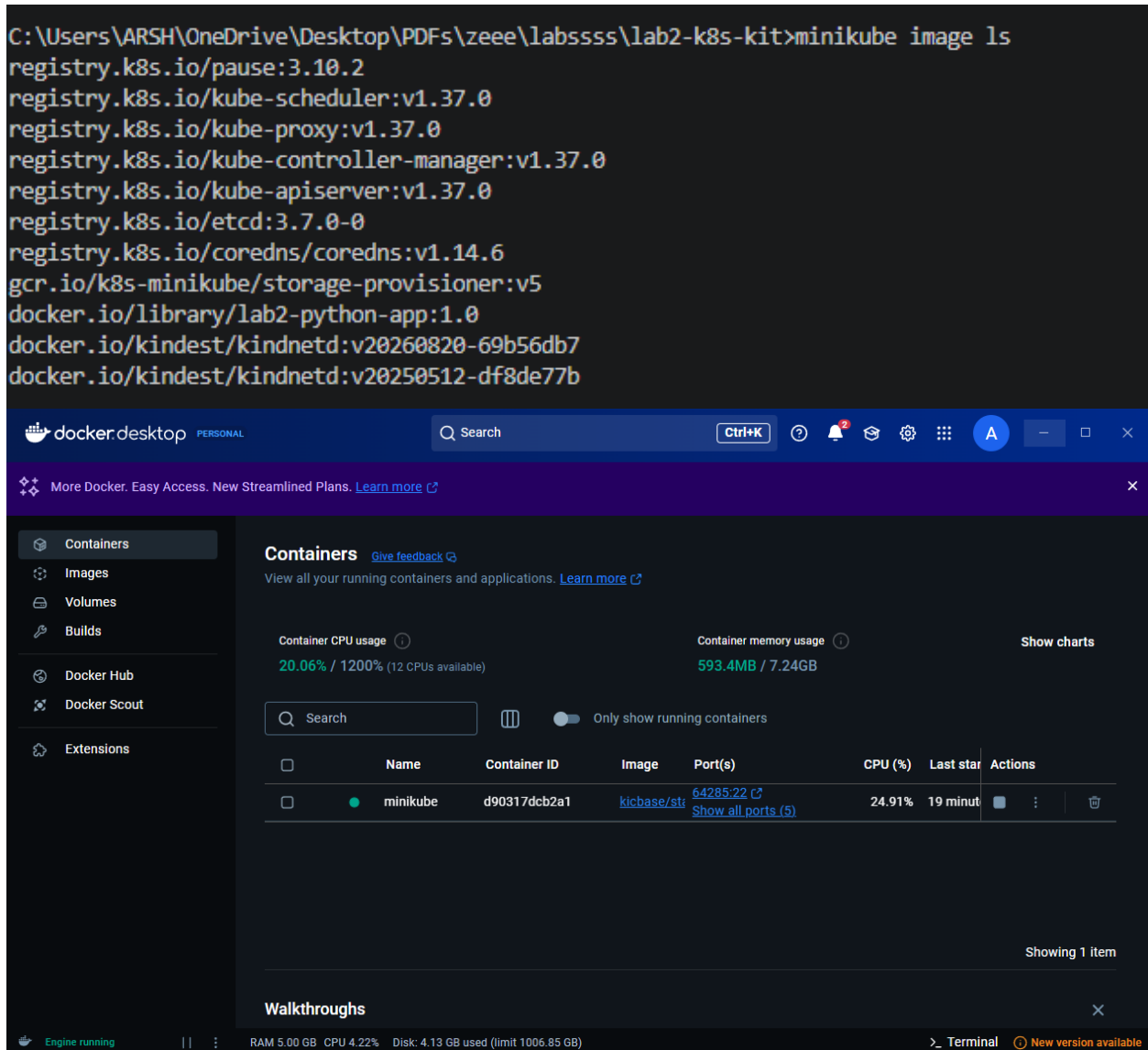


Figure 2 — `lab2-python-app:1.0` is present in the Minikube node's image store.

Step 3 — Write the Deployment and Service manifests

Both manifests are stored in the `k8s/` folder of the repository. The Deployment (`k8s/deployment.yaml`) declares the desired state of the application:

- **replicas: 3** — three identical pods are kept running at all times.
- **selector / labels (app: lab2-app)** — tie the pods to the Deployment and, later, to the Service.
- **strategy: RollingUpdate** — pods are replaced gradually on updates (used in Task 6).
- **readinessProbe and livenessProbe on /health** — a pod only receives traffic once `/health` answers, and is restarted if it stops answering.
- **resources** — small CPU and memory requests and limits for each pod.

`k8s/deployment.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: lab2-app
  labels:
    app: lab2-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: lab2-app
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: lab2-app
    spec:
      containers:
        - name: lab2-app
          image: lab2-python-app:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 5000
          readinessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 3
            periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 10
            periodSeconds: 10
          resources:
            requests:
              cpu: 50m
```

```
memory: 64Mi
limits:
  cpu: 250m
  memory: 128Mi
```

The Service (k8s/service.yaml) gives the pods one stable address. It selects pods by the app: lab2-app label and forwards NodePort 30080 (service port 80) to container port 5000.

k8s/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: lab2-app-svc
  labels:
    app: lab2-app
spec:
  type: NodePort
  selector:
    app: lab2-app
  ports:
    - port: 80
      targetPort: 5000
      nodePort: 30080
```

Step 4 — Apply the manifests

Both files are applied with a single command, and the rollout is watched until all replicas are ready.

```
kubectl apply -f k8s/
kubectl rollout status deployment/lab2-app
```

```

C:\Users\ARSH\OneDrive\Desktop\PDFs\zeee\labssss\lab2-k8s-kit>minikube image ls
registry.k8s.io/pause:3.10.2
registry.k8s.io/kube-scheduler:v1.37.0
registry.k8s.io/kube-proxy:v1.37.0
registry.k8s.io/kube-controller-manager:v1.37.0
registry.k8s.io/kube-apiserver:v1.37.0
registry.k8s.io/etcd:3.7.0-0
registry.k8s.io/coredns/coredns:v1.14.6
gcr.io/k8s-minikube/storage-provisioner:v5
docker.io/library/lab2-python-app:1.0
docker.io/kindest/kindnetd:v20260820-69b56db7
docker.io/kindest/kindnetd:v20250512-df8de77b

C:\Users\ARSH\OneDrive\Desktop\PDFs\zeee\labssss\lab2-k8s-kit>minikube image load lab2-python-app:1.0

C:\Users\ARSH\OneDrive\Desktop\PDFs\zeee\labssss\lab2-k8s-kit>minikube image ls
registry.k8s.io/pause:3.10.2
registry.k8s.io/kube-scheduler:v1.37.0
registry.k8s.io/kube-proxy:v1.37.0
registry.k8s.io/kube-controller-manager:v1.37.0
registry.k8s.io/kube-apiserver:v1.37.0
registry.k8s.io/etcd:3.7.0-0
registry.k8s.io/coredns/coredns:v1.14.6
gcr.io/k8s-minikube/storage-provisioner:v5
docker.io/library/lab2-python-app:1.0
docker.io/kindest/kindnetd:v20260820-69b56db7
docker.io/kindest/kindnetd:v20250512-df8de77b

C:\Users\ARSH\OneDrive\Desktop\PDFs\zeee\labssss\lab2-k8s-kit>kubectl apply -f k8s/
deployment.apps/lab2-app created
service/lab2-app-svc created

C:\Users\ARSH\OneDrive\Desktop\PDFs\zeee\labssss\lab2-k8s-kit>kubectl rollout status deployment/lab2-app
Waiting for deployment "lab2-app" rollout to finish: 0 of 3 updated replicas are available...
Waiting for deployment "lab2-app" rollout to finish: 1 of 3 updated replicas are available...
Waiting for deployment "lab2-app" rollout to finish: 2 of 3 updated replicas are available...
deployment "lab2-app" successfully rolled out

```

Figure 3 — `kubectl apply` creates the Deployment and the Service; rollout status reports the Deployment successfully rolled out.

Step 5 — Verify the running pods

The Deployment, its pods, the Service and the Service's endpoints are listed. The endpoints list shows one pod IP per replica, which confirms that the Service selector matches the pods.

```

kubectl get deployments
kubectl get pods -o wide
kubectl get svc lab2-app-svc
kubectl get endpoints lab2-app-svc

```

```
C:\Users\ARSH\OneDrive\Desktop\PDFs\zeee\labssss\lab2-k8s-kit>kubectl get deployments
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
lab2-app  3/3     3            3           14s

C:\Users\ARSH\OneDrive\Desktop\PDFs\zeee\labssss\lab2-k8s-kit>kubectl get pods -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
lab2-app-598fbbd44b-9hh77  1/1     Running    0          14s   10.244.0.5    minikube   <none>           <none>
lab2-app-598fbbd44b-9pj54  1/1     Running    0          14s   10.244.0.4    minikube   <none>           <none>
lab2-app-598fbbd44b-sjzf6  1/1     Running    0          14s   10.244.0.6    minikube   <none>           <none>

C:\Users\ARSH\OneDrive\Desktop\PDFs\zeee\labssss\lab2-k8s-kit>kubectl get svc lab2-app-svc
NAME      TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)    AGE
lab2-app-svc  NodePort    10.108.173.107 <none>        80:30080/TCP 15s

C:\Users\ARSH\OneDrive\Desktop\PDFs\zeee\labssss\lab2-k8s-kit>kubectl get endpoints lab2-app-svc
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME      ENDPOINTS                               AGE
lab2-app-svc  10.244.0.4:5000,10.244.0.5:5000,10.244.0.6:5000 17s
```

Figure 4 — The Deployment shows 3/3 ready; all three pods are Running with 1/1 containers ready; the Service exposes 80:30080/TCP and lists three endpoints.

Step 6 — Access the application through the Service

With the Docker driver on Windows, the node IP is not directly reachable from the host, so minikube service is used to open a tunnel to the NodePort service. It prints a local URL and must be left running in its own terminal.

```
minikube service lab2-app-svc --url
```

In a second terminal, the application is called through that URL (curl.exe is used because curl is an alias for Invoke-WebRequest in PowerShell). Replace <port> with the port printed above.

```
curl.exe -s http://127.0.0.1:<port>/
curl.exe -s http://127.0.0.1:<port>/health
```

```
C:\Users\ARSH\OneDrive\Desktop\PDFs\zeee\labssss\lab2-k8s-kit>kubectl apply -f k8s/
deployment.apps/lab2-app unchanged
service/lab2-app-svc unchanged

C:\Users\ARSH\OneDrive\Desktop\PDFs\zeee\labssss\lab2-k8s-kit>kubectl rollout status deployment/lab2-app
deployment "lab2-app" successfully rolled out

C:\Users\ARSH\OneDrive\Desktop\PDFs\zeee\labssss\lab2-k8s-kit>kubectl get deployments
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
lab2-app  3/3     3            3           4m7s

C:\Users\ARSH\OneDrive\Desktop\PDFs\zeee\labssss\lab2-k8s-kit>kubectl get pods -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
lab2-app-598fbbd44b-9hh77  1/1     Running    0          4m7s   10.244.0.5    minikube   <none>           <none>
lab2-app-598fbbd44b-9pj54  1/1     Running    0          4m7s   10.244.0.4    minikube   <none>           <none>
lab2-app-598fbbd44b-sjzf6  1/1     Running    0          4m7s   10.244.0.6    minikube   <none>           <none>

C:\Users\ARSH\OneDrive\Desktop\PDFs\zeee\labssss\lab2-k8s-kit>kubectl get svc lab2-app-svc
NAME      TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)    AGE
lab2-app-svc  NodePort    10.108.173.107 <none>        80:30080/TCP 4m7s
```

Figure 5 — The application answers through the Service; the / response reports the hostname of the pod that served the request, and /health returns {"status":"ok"}.

Result

The Flask application was deployed to Minikube using a Deployment (3 replicas, health probes, resource limits) and a NodePort Service. All pods reached the Running state with 1/1 containers ready, the Service listed one endpoint per pod, and the application responded on / and /health through the Service. Task 5 is complete.